

```
In [ ]: # CHURN ANALYSIS PROJECT MADE BY ISHIKA
```

```
In [1]: # import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

```
In [2]: #1. import database/data
```

```
In [3]: conn = sqlite3.connect('customer_churn.db')

sql_query = """
select name
from sqlite_master
where type='table'
"""

tables = pd.read_sql(sql_query , conn)

#create dataframe for each table
for table_name in tables['name']:
    df= pd.read_sql(f"select * from {table_name}",conn)
    globals()[f"df_{table_name}"]=df
    print (f"created dataframe: df_{table_name}")

conn.close()
```

created dataframe: df_db_customer
created dataframe: df_db_subscription
created dataframe: df_db_support

```
In [4]: #print table names and column names
conn=sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print (f"\nTable Name: {table_name}")
    #get column information
    columns_query = f"PRAGMA table_info({table_name});"
    columns= pd.read_sql(columns_query , conn)
    print ("columns:")
    print (columns['name'].tolist())

    #close connection
conn.close()
```

Table Name: db_customer

columns:

['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name: db_subscription

columns:

['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score']

Table Name: db_support

columns:

['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']

In [5]: `# df_db_customer.head()
df_db_customer.tail()`

Out[5]:

	customerid	name	country	state	gender	dob	interests	pincode
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

In [6]: `df_db_customer.info()`

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   customerid  21 non-null    str
1   name        21 non-null    str
2   country     18 non-null    str
3   state       21 non-null    str
4   gender      21 non-null    str
5   dob         21 non-null    str
6   interests   4 non-null     str
7   pincode     0 non-null     object
dtypes: object(1), str(7)
memory usage: 1.4+ KB
```

In [7]: `# a. rename column - name
b. drop columns - interest and pincode`

```
# c. change data type - dob  
# d. data standardization - gender  
# e. fix missing values - country
```

```
In [8]: # a. rename col - name  
  
df_db_customer.rename(columns = {'name' : 'customer_name'}, inplace = True)  
df_db_customer
```

Out[8]:

	customerid	customer_name	country	state	gender	dob	interests	pin
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	1
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	1
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	1
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	1
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	1
5	0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00	NaN	1
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00	NaN	1
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00	NaN	1
8	0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00	NaN	1
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00	NaN	1
10	0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00	NaN	1
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00	NaN	1
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00	NaN	1
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00	NaN	1
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00	NaN	1

	customerid	customer_name	country	state	gender	dob	interests	pincode
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00	job	110002
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	791005
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	302001
18	0022-TJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	500001
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	201002
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	411004

In [9]: *# b. drop column interest and pincode*

```
df_db_customer.drop(df_db_customer.columns[-2:], axis=1)
                    #[6:]

df_db_customer.drop(columns=['interests', 'pincode'], inplace = True)
```

In [10]: *# c. change daTA TYPE DOB*

```
df_db_customer['dob']=pd.to_datetime(df_db_customer['dob'])
df_db_customer
```

Out[10]:

	customerid	customer_name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05
5	0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14
8	0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29
10	0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06

```
In [11]: # d. data standardization - gender
df_db_customer['gender'].unique()

#df_db_customer['gender'] = df_db_customer['gender'].replace({'Men' : 'MaLe', 'Wo
```

```
Out[11]: <StringArray>
['Male', 'Female', 'Women', 'Men']
Length: 4, dtype: str
```

```
In [12]: # fixing missing values - country

df_db_customer[df_db_customer['country'].isna()]
```

Out[12]:

	customerid	customer_name	country	state	gender	dob
5	0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10
8	0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01

In [13]:

```
#[df_db_customer['country'].fillna('nahi pata')]  
  
df_db_customer[['country','state']]  
  
#country and state - unique value pair  
  
state_country_mapping = df_db_customer.dropna(subset = ['country']).set_index('s  
df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['sta
```

In [14]:

```
df_db_customer[['country','state']]
```

Out[14]:

	country	state
0	India	Maharashtra
1	India	Karnataka
2	India	Delhi
3	India	Nagaland
4	India	Delhi
5	India	Delhi
6	India	Meghalaya
7	India	Rajasthan
8	Nepal	Kathmandu
9	Nepal	Kathmandu
10	India	Maharashtra
11	India	Karnataka
12	India	Telangana
13	India	Meghalaya
14	India	Uttar Pradesh
15	India	Delhi
16	India	Meghalaya
17	India	Rajasthan
18	India	Telangana
19	India	Uttar Pradesh
20	India	Maharashtra

```
In [15]: df_db_subscription.head()  
#df_db_subscription.tail()  
#df_db_subscription.info()
```

Out[15]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	




```
In [16]: date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date' ]

df_db_subscription[date_col]=df_db_subscription[date_col].apply(pd.to_datetime)

df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customerid                           21 non-null     str
1   subscription_start_date               21 non-null     datetime64[us]
2   subscription_type                     21 non-null     str
3   renewal_date                         21 non-null     datetime64[us]
4   plan_type                            21 non-null     str
5   contract_type                        21 non-null     str
6   cancellation_date                     6 non-null      datetime64[us]
7   cancellation_reason                   6 non-null      str
8   monthly_charges                      21 non-null     float64
9   cltv                                 21 non-null     int64
10  churn_score                           21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 1.9 KB
```

```
In [17]: #df_db_support.head()
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid             9 non-null     str
1   complaint_date         9 non-null     str
2   escalations            9 non-null     str
3   csat_score             9 non-null     int64
4   col_1                  0 non-null     object
5   comment                 4 non-null     str
dtypes: int64(1), object(1), str(4)
memory usage: 564.0+ bytes
```

```
In [18]: df_db_support.drop(columns = ['col_1', 'comment'], inplace=True)
```

```
In [19]: df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])

df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid             9 non-null     str
1   complaint_date         9 non-null     datetime64[us]
2   escalations            9 non-null     str
3   csat_score             9 non-null     int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 420.0 bytes
```

In [20]: *# feature engineering and data analysis*

In [21]: *#creating new column using existing column - churn flag*

```
df_db_subscription['churn_flag']=np.where(df_db_subscription['cancellation_date']
df_db_subscription.head()
```

Out[21]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
--	------------	-------------------------	-------------------	--------------	-----------	------

0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
---	------------	------------	----------	------------	----------	--

1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
---	------------	------------	------	------------	---------	--

2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
---	------------	------------	---------	------------	-------	--

3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
---	------------	------------	------	------------	---------	--

4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	
---	------------	------------	----------	------------	----------	--



In [22]: *#first fix support table duplicates then merge*

```
df = (df_db_subscription
      .merge(df_db_customer, on = 'customerid', how = 'left')
      .merge(df_db_support, on = 'customerid', how = 'left'))
```

In [23]: df_db_subscription.shape

Out[23]: (21, 12)

In [24]: *#df.head*

```
df.shape
```

Out[24]: (23, 20)

In [25]: df_db_subscription['customerid'].nunique()

Out[25]: 21

In [26]: df_db_customer['customerid'].nunique()

Out[26]: 21

In [27]: df_db_support['customerid'].nunique()

Out[27]: 7

In [28]: df_db_support['customerid'].size

Out[28]: 9

In [29]: df_db_support.head()

Out[29]:

	customerid	complaint_date	escalations	csat_score
0	0003-MKNFE	2024-08-28	N	60
1	0003-MKNFE	2024-08-28	Y	10
2	0013-EXCHZ	2024-01-20	Y	20
3	0013-MHZWF	2025-03-18	N	90
4	0013-SMEOE	2024-11-01	N	30

In [30]: `df_db_support['complaint_count']=df_db_support.groupby('customerid')['customerid'`
`df_db_support= df_db_support.sort_values('complaint_date').drop_duplicates('cust`

In [31]: `df_db_support['customerid'].size`

Out[31]: 7

In [32]: `#merge`
`df = (df_db_subscription`
 `.merge(df_db_customer, on = 'customerid', how = 'left')`
 `.merge(df_db_support, on = 'customerid', how = 'left'))`

In [33]: `df.shape`

Out[33]: (21, 21)

In [34]: `df.to_csv('exported_churn_data.csv', index=False)`

In []:

In [35]: `#Data Analysis`

In [36]: `# 1. churn Rate`

In [37]: `df.columns`

Out[37]: Index(['customerid', 'subscription_start_date', 'subscription_type',
'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
dtype='str')

In [38]: `churn_rate = df['churn_flag'].mean()*100`
`print ('churn Rate = ', round(churn_rate,2),"%")`

churn Rate = 28.57 %

In [39]: `#2. RETENTION RATE`
`retention_rate = 100 - churn_rate`
`print ('retention Rate = ', round(retention_rate,2),"%")`

retention Rate = 71.43 %

```
In [40]: #3. churn by plan type
churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).
print (churn_by_plan)
```

```
plan_type  churn_rate_pct
0    Basic             60.00
1    Premium           14.29
2    Standard          22.22
```

```
In [41]: #4a. churn by state+sum(revenue)& count of users -- home work
#4b. churn by subscription type + sum(revenue) & count of users
```

```
In [42]: # 5. ARPU - average revenue per users
arpu=df['monthly_charges'].mean()
print ('ARPU =' , round(arpu,2))
```

ARPU = 18.85

```
In [43]: # 6. Avg tenure
today = pd.Timestamp.today()

df['tenure_days'] = np.where(
    df['cancellation_date'].notna(),
    (df['cancellation_date'] - df['subscription_start_date']).dt.days,
    (today - df['subscription_start_date']).dt.days
)
avg_tenure = df['tenure_days'].mean()
print('Avg Tenure (days) = ' , round(avg_tenure),0)
```

Avg Tenure (days) = 1513 0

```
In [44]: # 7. revenue at risk - revenue lost from churned users

revenue_at_risk = df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
print('revenue at risk (Rs. Lakhs)=', revenue_at_risk)
```

revenue at risk (Rs. Lakhs)= 73.94

```
In [45]: # 8. Escalation rate
escalation_rate = (df['escalations'] == 'Y').mean()*100
print ('Escalation rate=' , round(escalation_rate,2), '%')
```

Escalation rate= 19.05 %

```
In [46]: # 9. Avg complaint Rate per user
avg_complaints=df['complaint_count'].sum()/df['customerid'].nunique()
print ('avg complaints per user= ', round(avg_complaints,2))
```

avg complaints per user= 0.43

```
In [47]: # 10. correlation escalation vs churn

df['escalations'] = np.where(df['escalations'] == 'Y',1,0) # encoding string to in

corr_df = df[['escalations', 'churn_flag']].dropna()

correlation = corr_df['escalations'].corr(df['churn_flag'])
print ('correlation bw escalation vs churn=', round(correlation,2))
```

correlation bw escalation vs churn= 0.77

In [48]: *#11 . churn risk = create a column using existing column*

```
conditions = [
    (df['churn_score']<50),
    (df['churn_score']>= 50) & (df['churn_score']<70),
    (df['churn_score']>= 70)]
choices = ['low','med','high']
df['churn_risk']= np.select(conditions,choices,default= 'unknown')
```

In [49]: df.head()

Out[49]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
--	------------	-------------------------	-------------------	--------------	-----------	------

0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
---	------------	------------	----------	------------	----------	--

1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
---	------------	------------	------	------------	---------	--

2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
---	------------	------------	---------	------------	-------	--

3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
---	------------	------------	------	------------	---------	--

4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	
---	------------	------------	----------	------------	----------	--

5 rows × 23 columns



In []:

In []:

In [50]: *#4. visualisation using Matplotlib*

In [78]: df_visual = df.copy()

In [79]: *#df_visual.shape*
df_visual.columns

Out[79]: Index(['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob', 'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'tenure_days', 'churn_risk'], dtype='str')

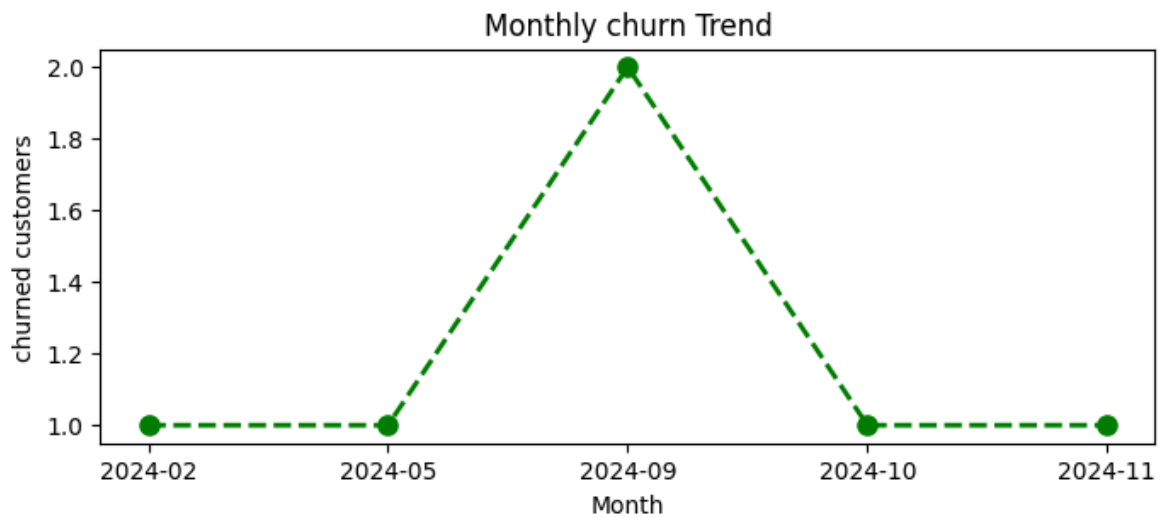
In [53]: *#4.1 Monthly churn trend (time series KPI)*

```
import matplotlib.pyplot as plt
```

```
df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')
```

```
churn_trend=df_visual[df_visual['churn_flag'] ==1].groupby('cancellation_month')
```

```
plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str),churn_trend.values, color='green', marker
plt.title('Monthly churn Trend')
plt.xlabel('Month')
plt.ylabel('churned customers')
plt.show()
```



```
In [54]: # 4.2 churn by plan type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

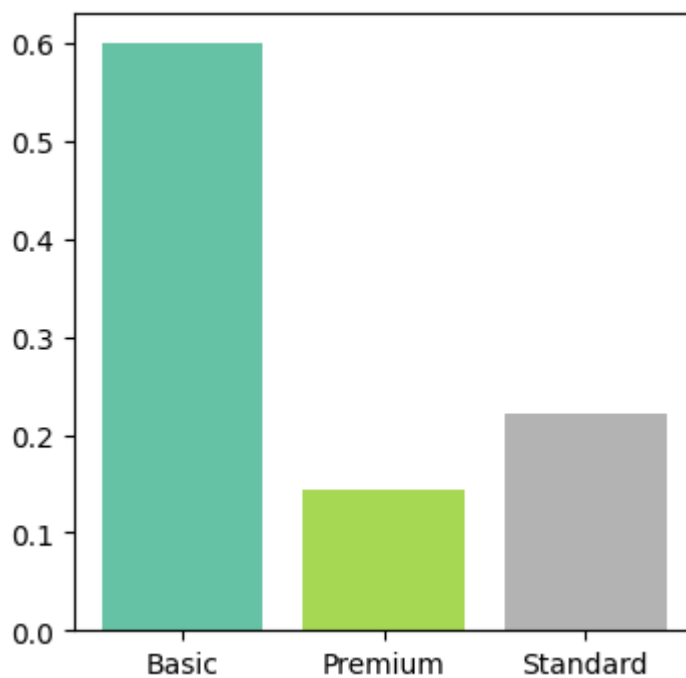
#colors = ['yellow' , 'purple' , 'blue']

colors = plt.cm.Set2(np.linspace(0,1,len(churn_plan)))

plt.figure(figsize=(4,4))

plt.bar( churn_plan.index, churn_plan.values , color = colors)

plt.show()
```



```
In [55]: # 4.3 churn by state
churn_plan = df_visual.groupby('state')['churn_flag'].mean()
```

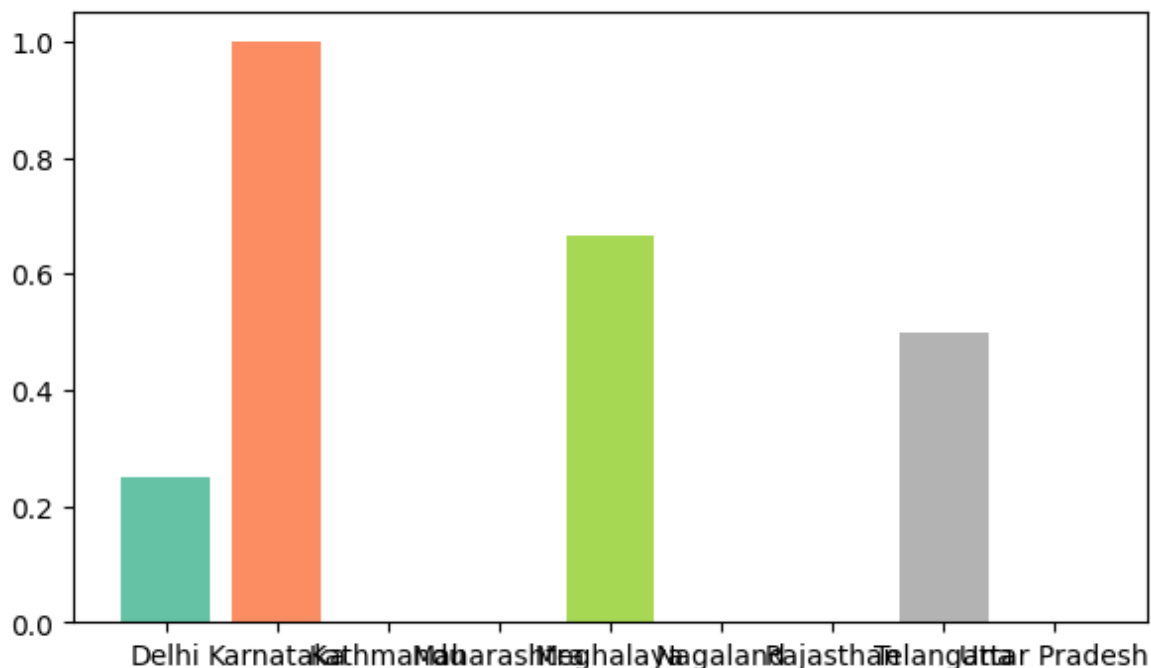
```
#colors = ['yellow' , 'purple' , 'blue']

colors = plt.cm.Set2(np.linspace(0,1,len(churn_plan)))

plt.figure(figsize=(7,4))

plt.bar( churn_plan.index, churn_plan.values , color = colors)

plt.show()
```



In []:

In [56]: *# visualizatio using seaborn*

In [77]: *# encoding - convert string to numeric so that we can find corr bw features*
df_visual.columns

Out[77]: Index(['customerid', 'subscription_start_date', 'subscription_type',
'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
'complaint_date', 'escalations', 'csat_score', 'complaint_count',
'tenure_days', 'churn_risk', 'cancellation_month'],
dtype='str')

In [58]: df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score',
'churn_flag', 'escalations', 'churn_risk']].head()

In [59]: *# Heatmap (correlation matrix)*

df_encoded.head()

Out[59]:

	plan_type	contract_type	churn_score	churn_flag	escalations	churn_risk
0	Standard	Annual	12	0	0	low
1	Premium	Annual	91	1	1	high
2	Basic	Monthly	34	0	0	low
3	Premium	Annual	8	0	0	low
4	Standard	Monthly	88	1	1	high

In [60]: `import warnings`
`warnings.filterwarnings("ignore")`

In [61]: *# incorrect method of encoding - as numbers are not assigned based on priority*

```

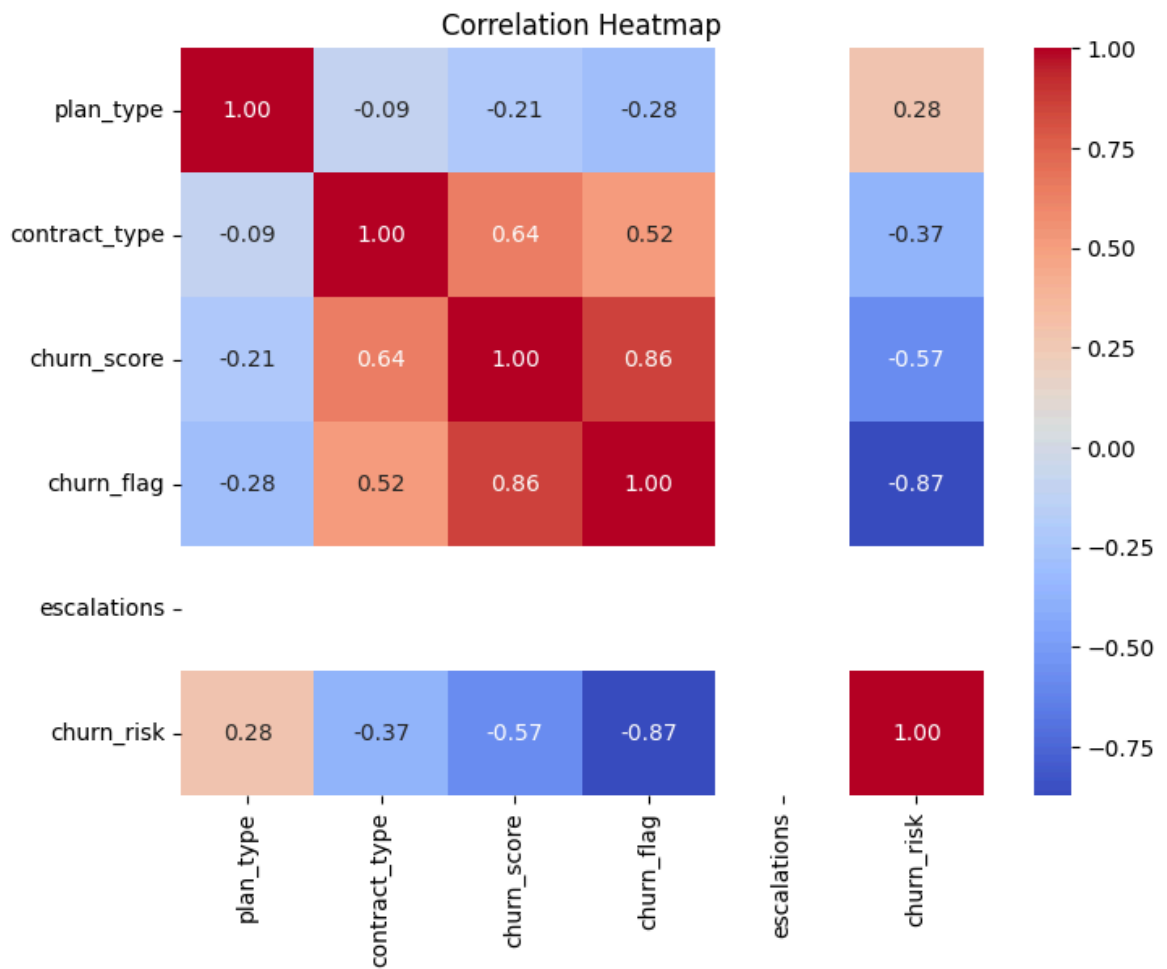
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score',
                        'churn_flag', 'escalations', 'churn_risk']]
df_encoded['escalations'] = df_encoded['escalations'].fillna('N') # treat NaN as
df_encoded['escalations'] = df_encoded['escalations'].map({'Y': 1, 'N': 0}).fill
df_encoded['escalations'] = df_encoded['escalations'].astype(str).str.upper().st
categorical_cols = ['plan_type', 'contract_type', 'churn_risk']

for col in categorical_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes

import seaborn as sns
import matplotlib.pyplot as plt

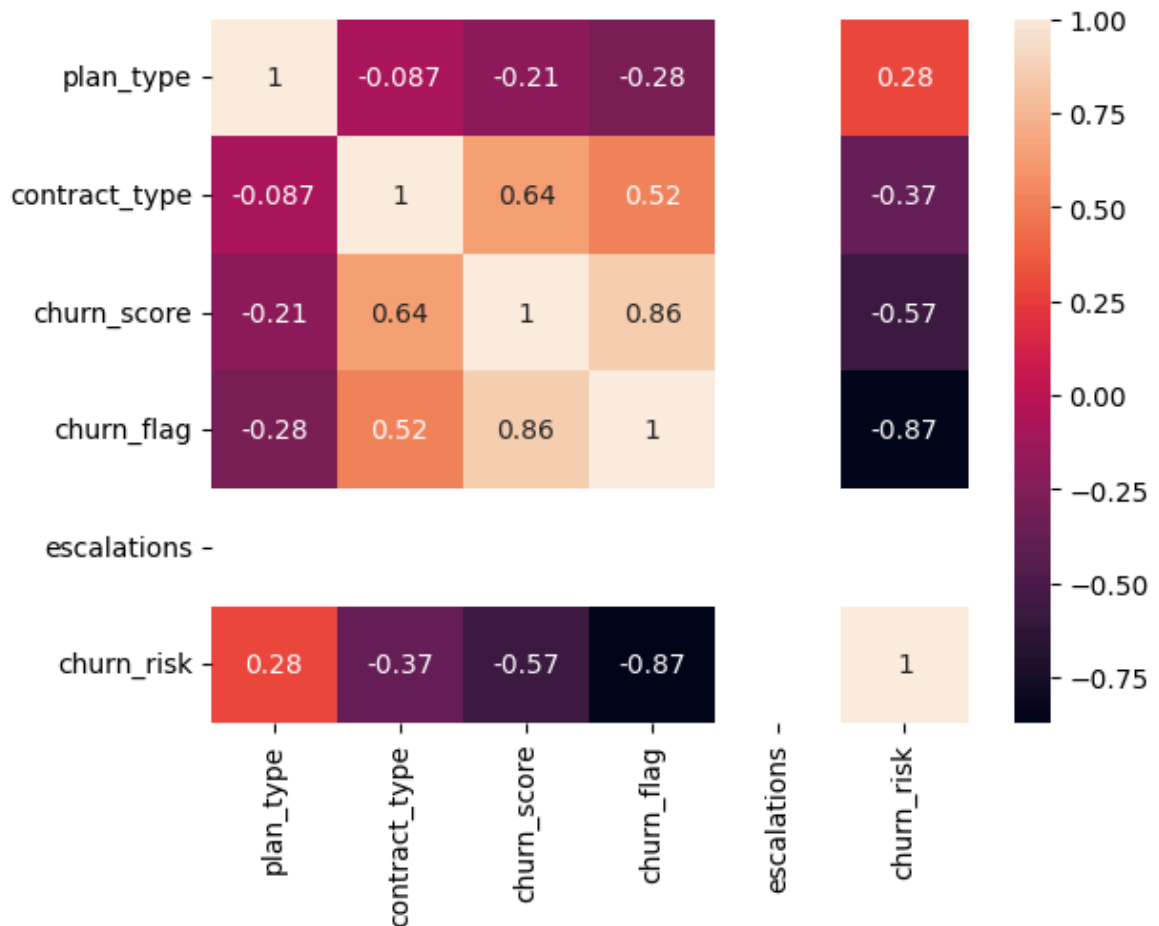
plt.figure(figsize=(8,6))
sns.heatmap(df_encoded.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()

```

```
In [62]: # heatmap (correlation matrix)
sns.heatmap(df_encoded.corr() , annot = True)
```

```
Out[62]: <Axes: >
```



```
In [63]: df_visual['churn_risk'].unique()
```

```
Out[63]: <StringArray>
['low', 'high', 'med']
Length: 3, dtype: str
```

```
In [64]: # correct method of encoding - as numbers are not assigned based on priority

df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score',
                        'churn_flag', 'escalations', 'churn_risk']]

order_mappings = {
    'plan_type': ['Basic', 'Standard', 'Premium'],
    'contract_type': ['Monthly', 'Anually'],
    'churn_risk': ['low', 'medium', 'high']
}

for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col], categories = order, order
```

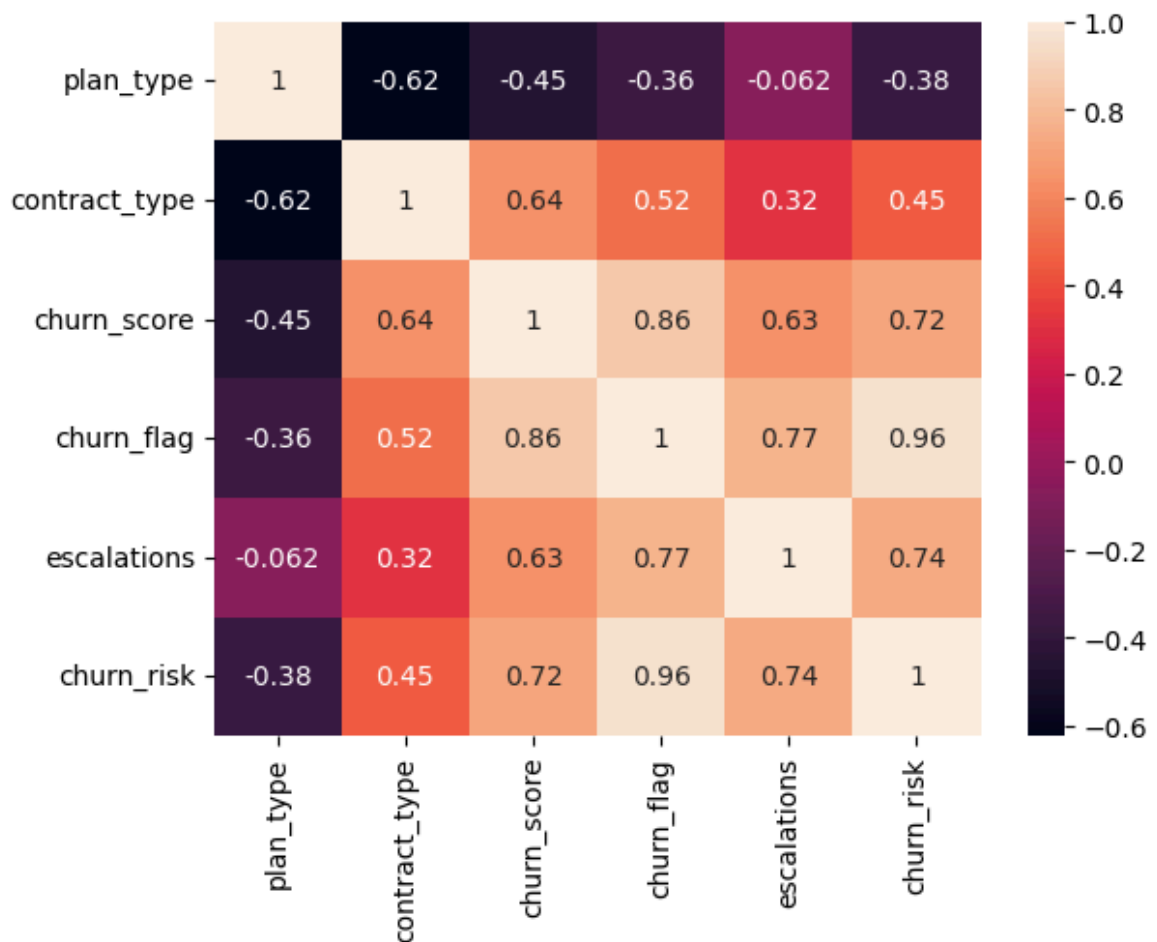
```
In [65]: df_encoded.head()
```

Out[65]:

	plan_type	contract_type	churn_score	churn_flag	escalations	churn_risk
0	1	-1	12	0	0	0
1	2	-1	91	1	1	2
2	0	0	34	0	0	0
3	2	-1	8	0	0	0
4	1	0	88	1	1	2

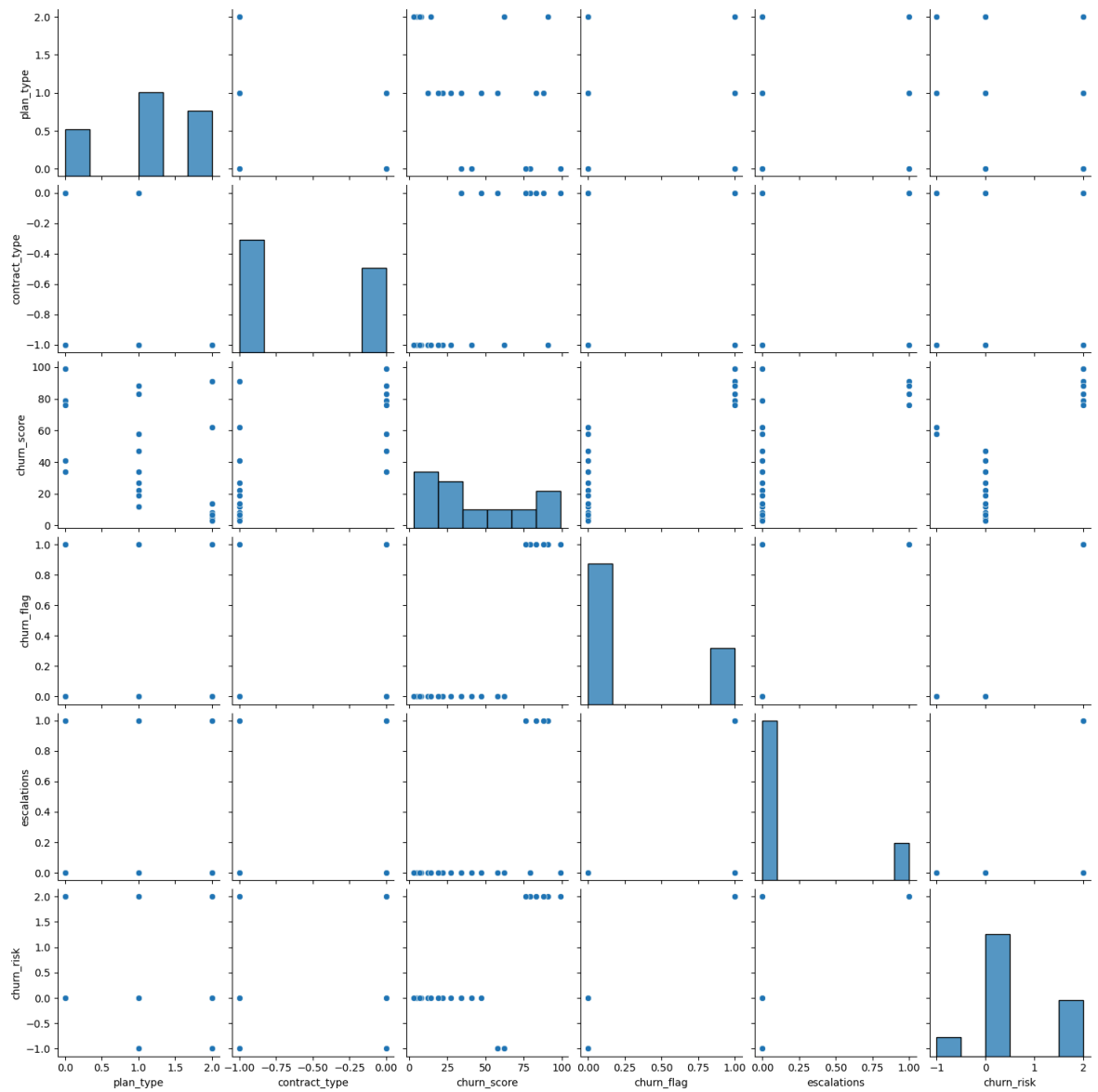
In [66]: `sns.heatmap(df_encoded.corr() , annot = True)`

Out[66]: <Axes: >



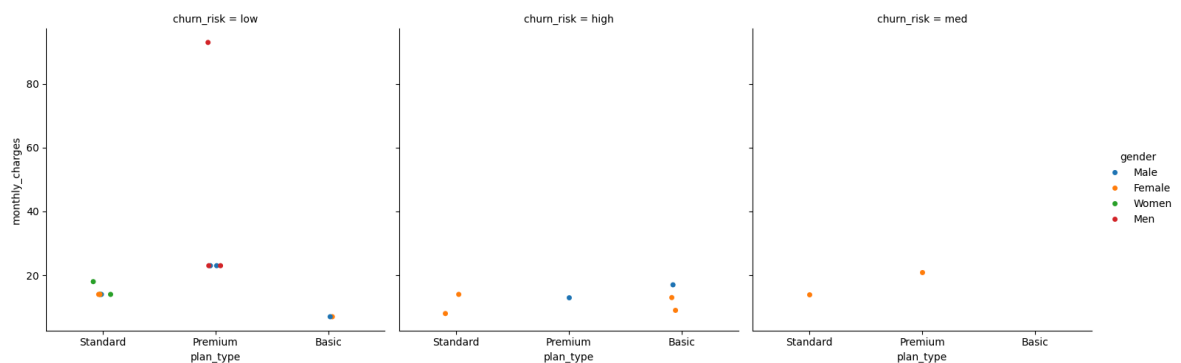
In [67]: `#pair plot`
`sns.pairplot(df_encoded)`

Out[67]: <seaborn.axisgrid.PairGrid at 0x22e7f225a90>



```
In [69]: # catplot / facegrid chart - multi-dim comparision
sns.catplot(data = df_visual,
            x= 'plan_type',
            y= 'monthly_charges',
            hue = 'gender',
            col= 'churn_risk')
```

```
Out[69]: <seaborn.axisgrid.FacetGrid at 0x22e7f2257f0>
```



```
In [82]: #pivot table
pd.pivot_table(
    df_visual,
```

```

values=['churn_flag', 'monthly_charges', 'customerid'],
index='plan_type',
aggfunc={
    'monthly_charges': 'sum',
    'customerid' : 'nunique',
    'churn_flag' : 'mean'
}
)

```

Out[82]:

	churn_flag	customerid	monthly_charges
plan_type			
Basic	0.600000	5	52.95
Premium	0.142857	7	218.93
Standard	0.222222	9	123.91

```

In [ ]: pd.pivot_table(
    df_visual,
    values=['churn_flag', 'monthly_charges', 'customerid'],
    index='plan_type',
    aggfunc={
        'monthly_charges': 'sum',
        'customerid' : 'nunique',
        'churn_flag' : 'mean'
    }
)

```

```

In [ ]: # working with sql in python (pandas)

```

```

In [88]: # create db in sql
conn=sqlite3.connect('test_database.sqlite')
# table details
conn.execute("create table users ( first_name Text, country Text, budget integer
#commit and save
conn.commit()

```

```

-----
OperationalError                                Traceback (most recent call last)
Cell In[88], line 4
      1 # create db in sql
      2 conn=sqlite3.connect('test_database.sqlite')
      3 # table details
----> 4 conn.execute("create table users ( first_name Text, country Text, budget
integer)")
      5
      6 #commit and save
      7 conn.commit()

OperationalError: table users already exists

```

```

In [92]: # insert data

cursor = conn.cursor()
#query
cursor.execute(

```

```

        """
        INSERT INTO USERS VALUES
            ('Madhav', 'India', 5000),
            ('Rishab', 'USA', 6000),
            ('Vishakha' , 'China', 7000)
        """
    )
    # commit and save
    conn.commit()
    print ("data inserted successfully")

```

data inserted successfully

```

In [93]: # check inserted data in table
conn = sqlite3.connect('test_database.sqlite')
query = """select * from users"""

df_results = pd.read_sql(query,conn)

df_results.head()

```

Out[93]:

	first_name	country	budget
0	Madhav	India	5000
1	Rishab	USA	6000
2	Vishakha	China	7000

```

In [95]: # aggregation
query = """
select country , sum(budget) as total_budget
from users
group by country
        """
df_agg = pd.read_sql(query,conn)
df_agg

```

Out[95]:

	country	total_budget
0	China	7000
1	India	5000
2	USA	6000

```

In [ ]: #close conn after use
conn.close

```